Predicting Central Carbon Coordinates from Amino Acid Sequences

Andreas Munck – Chalmers University of Technology

Introduction

Proteins enables the possibility of life to exist. Understanding their functions can save lives through, for example drug development. A protein's function is determined by its 3D structure. However, determining this structure experimentally is a costly process. Further, determining a protein's amino acid sequence is much faster and cheaper. Machine learning based models, such as Alphafold 2, make use of this, by predicting a protein's 3D structure from its amino acid (AA) sequence with near-experimental accuracy [1]. Still, Alphafold 2 is a complex large model and there are problems that require simpler models that can predict faster. One example is the problem of identifying antibodies in human blood, because of the very large number ($> 10^{12}$) of possible antibodies [2].

A protein is a chain of AAs. An AA consist of two parts. One part, called the backbone, is the same for all amino acids, one of these atoms is a carbon atom called the central carbon atom (C_α). The other part, called the side chain, differentiates the types of AAs, there are 20 different sidechains. **This project investigates the possibility of predicting the position of one atom, the central carbon atom (C_α), in each AA of a AA-chain, using neural networks.**

Data

The ids for the suitable, see Table 1, files containing the AA sequences and Cα-coordinates was downloaded through the *rcsb-api* Python library [3], which builds on the RCSB.org API [4]. The ids were then used to download the files from RCSB.org. Two datasets were gathered, by filtering on the attributes shown in Table 1.

Table 1: Attributes used to filter for suitable proteins.

Attribute	Condition
# chains in molecule	= 1
Resolution	≤ 3.5 Å
Polymer type	= "Protein"

The first and third attibutes are to make sure that the protein is connected. The second attribute is set to 3.5 Å since this is roughly the minimum distance between two central carbon atoms, in most cases.

When extracting the AA sequence from the files, the number of unknown AAs were checked. If the ratio exceeded 0.1 this sequence was not included in the dataset.

The *rcsb-api* was also used to filter for minimum length and maximum length of the AA sequence, forming two datasets seen in Table 2.

Table 2: Datasets used, distinguished by minimum and maximum AA sequence length.

Dataset label	Minimum length	Maximum length	# Samples
A	5	50	312
B	20	70	721

The increase of the minimum and maximum length in dataset B allowed for a larger dataset and more folded chains. The datasets were split with 90% to training and validation, with an internal split of 80% to training, and 10 % to the test .

It is hard to directly predict coordinates from an AA sequence, since the protein could have been rotated and translated in the experimental measurement. Therefore, the C_α-coordinates were used to calculate Euclidean distances for all pairs of AAs, to form distance matrices. However, even the relative coordinates of AAs are not fixed, due to thermal fluctuations, amongst other reasons. Therefore, it is reasonable to predict a probability distribution of the possible distances between AAs instead of by regression trying to find an exact distance. On instance of the model was trained on distance matrices, while two were trained on distrograms, distance histograms. The bin edges, in Å, were

$$0.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 7.0, 8.0, 9.0, 10.5, 13.0, 15.0, 17.0, 21.0, 25.0,$$

forming 16 bins.

Multi Sequence Alignment

Multi sequence alignment (MSA) is used to extract evolutionary information from AA sequences. Each AA sequence is compared with similar sequences. One way of extracting information can be to find places where mutations can have occurred. If one mutation changes the charge of an AA, a neighbouring AA can also mutate to compensate for this, showing that these are spatially close. In this model, the nine most similar sequences, in the dataset, were appended under each AA sequence, forming a matrix of shape $10 \times$ max length AA sequence.

The model, see Figure 6, used self-attention along rows and then columns for the matrices, to same memory and computation. The embedding dimension was 16 here for dataset B and 32 for dataset B. The mean outer product was then computed for each matrix. That is, each matrix was passed to two parallel linear layer, retaining dimension, and then the mean was taken along columns for the parallel matrices forming two 1D tensors. Continuing, the outer product was computed of these 1D tensors forming a matrix that could be added to the paired embedding.

Paired Embedding

Each AA sequence was passed to a transformer encoder with two transformer layer for dataset B and four for dataset A. In each layer there were 8 attention heads. The embedding dimension was 256. Therefore, it was projected down to match the dimension of the MSA.

Triangle Update

When the MSA and paired embeddings had been added they were passed to a triangle update mechanism. This is one of the core mechanisms used in Alphafold 2 [1]. The word triangle comes from using a third AA as reference point, to hopefully introduce geometrical constrain, such as the triangle inequality. The triangle updates can be visualized by Figure 1.

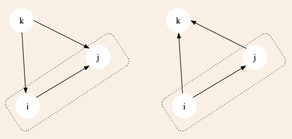

Figure 1: Triangle update in to the left and out to the right. The distance between one pair is compared to the distances to a reference point.

The triangle update sums over each pair ij in the embedding matrix and and compares the distance between i and j to the distances to a third AA k. Triangle update out looks for the k AA along rows while triangle update out looks along columns. To force the matrix to be symmetric, the average is taken between the upper and lower triangle before backpropagation. The loss function was mean squared error for distance matrices and crossentropy for distograms.

Distances to Coordinates

First an classical multidimensional scaling (MDS) was used to get an approximation of the coordinates that best fit the distance matrix. In short MDS transforms the square of the distance matrix so that the mean of all rows and columns are zero. Then it calculates the coordinates from the eigenvalues and eigenvectors of this transformed matrix. To further fine-tune the coordinates gradient descent, with LBFGS, was applied on the loss function L

$$L = \sum_{i=1}^{N}\sum_{j=1}^{N}\frac{1}{2}(D_{ij}^{\text{pred}} - ||\mathbf{X}_i - \mathbf{X}_j||)^2. \tag{1}$$

$\mathbf{X}$ is the coordinates to be optimized, D_{ij} are the elements of the distance matrix $\mathbf{D}$. This gave coordinates that best fitted the distance matrices but they could still be rotated and translated compared to the correct coordinates.

Results and Analysis

Firstly, the algorithm to convert distance matrices to coordinates almost works. Correct distance matrices from dataset B were used as a test. The root mean squared error (RMSE) after MDS and gradient descent RMSE and before or after rotation and translation can be seen in Table 3.

Table 3: Test of distance matrix to coordinates.

RMSE **D** before rotations and translation	RMSE **X** before rotations and translation	RMSE **X** after rotations and translation
$1.30 \cdot 10^{-4}$ Å	19.9 Å	3.8 Å

The 3.8 Å RMSE error after the translation and rotation is probably due to the coordinates after gradient descent being mirrored to the correct ones. One of the wrong prediction of coordinates from a correct distance matrix can be seen in Figure 2, where one can see that the structure is correct. Most of the predicted coordinates overlapped with the correct ones.

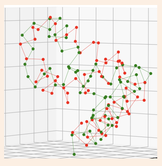

Figure 2: Predicted coordinates in red, predicted using a correct distance matrix. Correct coordinates in green.

Now the problem is almost reduced to finding a correct distance matrix. The results for the predicted distance matrix and coordinates using only AA sequences from the test set as inputs are shown in Table 4.

Table 4: The root mean square error (RMSE) in different stages of the model. $\mathbf{X}$ is before the rotations and translations and $\mathbf{X}_{\text{Aligned}}$ is afterwards.

Dataset	Accuracy Bins [%]	RMSE **D** [Å]	RMSE **X** [Å]	RMSE $\mathbf{X}_{\text{Aligned}}$ [Å]
A	Regression	4.90	12.3	7.30
A	21.6	6.44	12.8	7.43
B	35.6	8.13	19.9	12.2

First note that the model instances for the two models that were classification of bins were not tuned as thoroughly as the instance for regression. Dataset A also gives better results, partly because shorter proteins are on average smaller, but probably mostly because a lot of proteins in data set A did not fold. Therefore, the model easily learned to predict proteins that were further away in the sequence to be further away in 3D space. This is illustrated in Figure 3.

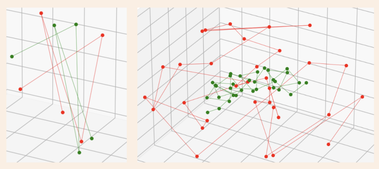

Figure 3: To the left a shorter AA sequence, with not that much folding. To the right a longer AA sequence with more folding. In green are the correct coordinates and in red the predicted. The data set was A and the model used distograms, that is it classified bins.

The model that trained on dataset B probably tried to learn the folding more, as seen in Figure 4.

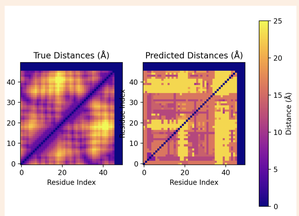

Figure 4: One distance matrix from prediction on dataset B. RMSE for this protein was 7.04 Å and the bin accuracy 29 %.

Some predictions on dataset B one long sequences were quite close, while others were far of. One example were it was close is show in Figure 5.

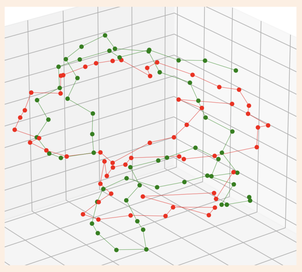

Figure 5: Correct coordinates in green and predicted in red.

Conclusions and Future Work

The model was not able to predict the central carbon atom of proteins, even if their AA sequence was relatively short. The procedure of predicting coordinates from distance matrices were successful, however no neural networks were used here. The problem lies in learning enough about the sequence to predict the distance matrices. The problem lies in the complexity of succesful methods such as the triangle update mechanism. For future work, one can try and implement trianlge attention as in ALphafold 2, Try and get better MSA from a larger database, or use template 3D coordinates from similar proteins.

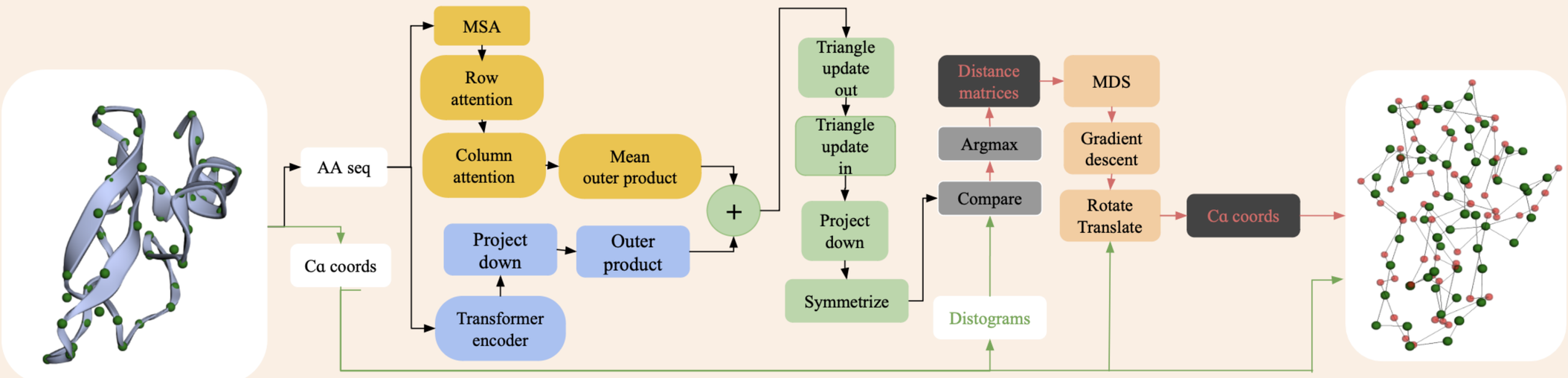

Figure 6: The multi-stage model used for predicting C_α from AA sequences. Note that the picture to the right is just noise added to the correct coordinates, not predicted coordinates.

References

[1] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, et al. Highly accurate protein structure prediction with alphafold. *Nature*, 596:583–589, 2021. Relevant because it introduces Alphafold 2, that can achieve near-experimental accuracy in protein 3D structure prediction.

[2] B. Briney, A. Inderbitzin, K. Bahl, and D. R. Burton. Commonality despite exceptional diversity in the baseline human antibody repertoire. *Nature*, 566:393–397, 2019. Relevant because it highligts a problem where a faster prediction model is needed.

[3] Dennis W. Piehl, Brinda Vallat, Ivana Truong, Habiba Morsy, Rusham Bhatt, Santiago Blaumann, Pratyoy Biswas, Yana Rose, Sebastian Bittrich, Jose M. Duarte, Joan Segura, Chunxiao Bi, Douglas Myers-Turnbull, Brian P. Hudson, Christine Zardecki, and Stephen K. Burley. rcsb-api: Python toolkit for streamlining access to rcsb protein data bank apis. *Journal of Molecular Biology*, 2025. Used for accessing protein data in this project.

[4] Yana Rose, Jose M. Duarte, Robert Lowe, Joan Segura, Chunxiao Bi, Charmi Bhikadiya, Li Chen, Alexander S. Rose, Sebastian Bittrich, Stephen K. Burley, and John D. Westbrook. Rcsb protein data bank: Architectural advances towards integrated searching and efficient access to macromolecular structure data from the pdb archive. *Journal of Molecular Biology*, 2020. Describes the API services used by rcsb-api.